



[Institution's Name]

A Project

Report On

"CBSE Database Management Software"

For

AISSCE 20XX Examination

As a part of the Computer Science Course(083)

Submitted By:

.

.

Under the Guidance of:

Certificate

This is to certify that the Project/ Dissertation entitled, CBSE Database Management Software, is a bonafide work done by Master/Miss ___ of Class ___, Session ___, in partial fulfillment of ___ and has been carried out under my direct supervision and guidance.

This report or a similar report on the topic has not been submitted for any other examination and does not form a part of any other course undergone by the candidate.

Signature of Teacher

Signature of Principal

Name: ___

Designation: ___

Signature of External Examiner

Name:

Examiner No:

Date:

Acknowledgment

As usual, a large number of people deserve my thanks for the help they provided to me for the preparation of this project.

First of all, I would like to thank my teacher ___ for his support during the preparation of this project. I am very thankful for his guidance.

I would also like to thank my family and friends for their genuine love and support during the course of the project.

At last, I would also like to show our sincere gratitude to everyone who directly or indirectly have lent their helping hand in this venture.

Name: ___

Class: ___

Section: ___

Class Roll Number: ___

CBSE Exam Roll Number:

Index

<u>Index</u>	<u>Topic</u>	<u>Page Number</u>
1	Certificate	3
2	Acknowledgment	4
3	Objective	7
4	Scope	8
5	Theoretical Background	10
6	System Planning	11
7	Methodology Adopted and Details of Hardware/ Software Used	14
8	Snippet of Source Code	16
9	Snippets of Test Run	33
10	User Manual	36
11	Brief Background of Organization	39
12	Reference	40
13	Appendix A and Appendix B	41 and 44

Objective

The primary objective of this project titled CBSE Database Management Software is to explore real-life database management concepts through a practical and comprehensive software system. The project aims to simulate the core functionalities of Central Board of Secondary Education's academic examination management platform by modeling affiliated institutions, courses, students, and examination results using a relational database.

The system enables users to view affiliated schools, generate examination admit cards, and retrieve final examination mark sheets for students appearing in secondary (Class X) and senior secondary (Class XII) examinations. These features are implemented to reflect how structured data is stored, processed, and retrieved in real-world database-driven applications.

To achieve these objectives, MySQL is used as the back-end database management system for structured data storage and integrity enforcement, Python is used as the primary programming language for data generation and application logic, and Python-Tkinter is used to provide a graphical user interface for user interaction. The project emphasizes modular program design, Python-MySQL connectivity, and the application of database constraints to ensure data consistency and validity.

Overall, the project aims not merely to replicate an existing system, but to demonstrate how real-life database management concepts can be understood and implemented through a working example.

Scope of the Project

What the Project Aims to Cover

The scope of this project includes the following functionalities:

- **Simulation** of an **academic examination management system** with a relational database back end.
- **Collection and processing** of **real affiliated school data**.
- **Generation and display** of **examination admit cards** for registered students.
- **Generation and display** of **subject-wise examination results and final mark sheets** for students.

What the Project Does Not Aim to Cover

- **Complete replication** of the official examination board website or its auxiliary services.
- **Administrative workflows** such as registration portals, payment systems, or result analytics dashboards.
- **Real-time data synchronization** with external systems.

Assumptions Made During the Project

- A new **database is created** for each academic year, named in the format **CBSE_EXAM_RESULT_YXXXX**, where XXXX represents the current year.
- The database consists of **four primary tables: COURSES, AFFILIATED_SCHOOLS, REGISTERED_STUDENTS, and EXAM_RESULTS**.
- A fixed number of **500,000 students** appear for the examinations, with the **ratio of Class XII to Class X students maintained at 3:2**.
- All registered **students** are assumed to have **appeared in all their examinations**, with **no absentees**.
- Each student is assigned exactly **seven subjects: five main subjects and two additional subjects**.

Limitations of the Project

- The number of affiliated schools is limited due to manual data collection, resulting in an unrealistically high number of students per school.
- Examination centres are not constrained to the student's district.
- Subject selection rules are simplified and do not fully reflect curriculum-specific language requirements.

Theoretical Background

This project is based on the relational database model, where data is organized into structured tables composed of rows and columns. Relationships between entities are maintained using primary keys and foreign keys to enforce referential integrity and consistency across the database.

Structured Query Language (SQL) is used to define schemas, enforce constraints, and retrieve data using joins across multiple related tables. The project relies on database constraints to maintain correctness of marks, uniqueness of identifiers, and validity of records.

MySQL is used as the relational database management system due to its support for constraints, efficient handling of large datasets, and compliance with standard SQL. Database constraints are employed extensively to ensure validity of marks, uniqueness of identifiers, and correctness of relational mappings.

Python is used as the application layer to generate data, validate records, and interact with the database through SQL queries. The Python-Tkinter library is used to develop a cross-platform graphical user interface. Additional libraries such as Pillow are used for image handling, Tabulate is used for structured tabular display within the GUI, Faker is used for realistic data generation, and MySQL Connector/Python is used for database connectivity.

The project uses the UV Python package manager to manage dependencies and maintain a reproducible development environment.

System Planning

The system was planned as a database-driven application with Python acting as the control layer and MySQL serving as the persistent storage layer. The database schema was designed before implementation to clearly define entities such as courses, affiliated schools, registered students, and examination results.

The project followed a phased development approach consisting of:

- requirements analysis
- system design
- implementation
- testing
- documentation

Project Directory Structure

The project followed a structured directory layout to separate source code, data, database schemas, documentation, and static assets.

The `src/` directory contains setup and parsing scripts used during initial data preparation, while the `modules/` directory contains reusable Python modules used across the project.

All raw and processed datasets are stored under the `data/` directory, with additional backups preserved for reference and recovery. Database schemas/ are isolated in the `schemas` directory to maintain clear separation between application logic and database design.

Documentation, reports, and references are stored in the `docs/` directory, while static resources such as images and logos are kept under `assets/`. This organization improves maintainability, readability, and scalability of the project.

The directory structure, after the successful completion of the project, is as follows:

```

.
├── assets
│   ├── CBSE_logo.png
│   ├── KVS_logo.jpeg
│   └── transparent.png
├── backup
│   ├── affiliated_schools.dat
│   ├── affiliated_schools.txt
│   ├── courses.dat
│   └── courses.txt
├── data
│   ├── raw_datasets
│   │   ├── affiliated_schools_raw_data
│   │   │   ├── 001.png
│   │   │   ├── 002.png
│   │   │   ├── 003.png
│   │   │   ├── 004.png
│   │   │   ├── 005.png
│   │   │   ├── ...
│   │   │   ├── 099.jpg
│   │   │   ├── 100.jpg
│   │   │   ├── 101.jpg
│   │   │   ├── 102.jpg
│   │   │   └── 103.png
│   │   ├── courses_raw_data
│   │   └── raw_data_for_reference
│   ├── grade_10_additional_courses_that_were_filtered_during_setup.txt
│   ├── grade_10_main_courses_that_were_filtered_during_setup.txt
│   ├── grade_12_additional_courses_that_were_filtered_during_setup.txt
│   ├── grade_12_main_courses_that_were_filtered_during_setup.txt
│   ├── pickled_affiliated_schools.txt
│   ├── pickled_courses_data.txt
│   ├── view_of_exam_results_generated_during_setup.txt
│   ├── tabular_data
│   │   ├── affiliated_schools.txt
│   │   └── courses.txt
│   ├── affiliated_schools.dat
│   └── courses.dat
├── docs
│   ├── CS_Project_Report.odt
│   ├── CS_Project_Report.pdf
│   ├── Curriculum_Secondary_2025-26.pdf
│   ├── Curriculum_Senior_Secondary_2025-26.pdf
│   ├── MySQL_Integer_Types.png
│   ├── notes.txt
│   └── references.txt
├── modules
│   └── connection_module.py

```

```
|   |   | functions.py
|   |   | __init__.py
|   | schemas
|   |   | database_schema.sql
|   | src
|   |   | forbidden_scripts
|   |   |   | manually_update_affiliated_schools_from_tabular_data.py
|   |   | parse_affiliated_school_data.py
|   |   | parse_courses.py
|   |   | set_configuration.py
|   |   | setup.py
|   | config.json
|   | main.py
|   | pyproject.toml
|   | README.md
|   | requirements.txt
|   | uv.lock
```

14 directories, 143 files

Methodology Adopted and Details of Hardware/ Software Used

Methodology Adopted

The project was developed using the following methodology:

- 1.Design of the relational database schema with appropriate keys and constraints.
- 2.Collection and preprocessing of affiliated school and course data.
- 3.Programmatic generation of student records and examination results at runtime.
- 4.Automated database setup and data insertion using Python scripts.
- 5.Retrieval and display of data through SQL queries integrated into a graphical interface.

Hardware Used

The software was developed and tested on a personal computer with the following specifications:

- Linux Mint 22.2 and Linux Mint 22.3 operating system
- 4 GB DDR3 RAM
- 512 GB storage
- Intel Core i5-3570K processor with integrated graphics

Additional testing was performed on Windows 10 and Windows 11 systems.

Software Used

- Python 3.14
- MySQL Server 8.0
- Python-Tkinter for GUI development
- UV Python package manager for dependency management
- Development and testing environments included PyCharm, Visual Studio Code, and Zed IDE

Snippet of Source Code

main.py

```
"""
Python program to access MySQL database.
"""

import atexit
import json
import sys
from pathlib import Path

import tkinter as tk

from PIL import Image, ImageTk
import tabulate

from modules import ConnectToMySQL

PATH_TO_ASSETS_DIR = Path() / "assets"
PATH_TO_CONFIG      = Path() / 'config.json'

ASSETS: list[Path] = [
    PATH_TO_ASSETS_DIR / 'CBSE_logo.png',
    PATH_TO_ASSETS_DIR / 'KVS_logo.jpeg'
]
MISSING_ASSETS = []

def check_for_missing_assets() -> bool:
    """
    :return: True if all assets exists, else False
    """

    for _asset in ASSETS:
        if not _asset.exists():
            MISSING_ASSETS.append(_asset.absolute().__str__())

    if MISSING_ASSETS:
        return True
    return False

if check_for_missing_assets():
    print(f"The following assets are missing, go fix them somehow:\n\n\t{'\n\t'.join(MISSING_ASSETS)}\n")
    print(
        f"Please go to my GitHub repo: ..., and from assets directory, download the missing images and\n"
        f"move them inside the following directory: {PATH_TO_ASSETS_DIR.absolute()}"
    )
```

```

        print("\nTill no fix, program will not continue to work.
Terminating script early...")
        sys.exit()

```

```

if not PATH_TO_CONFIG.exists():
    print(f"Config file doesn't exists at the following file path:
{PATH_TO_CONFIG.absolute()}")
    print("Run the set_configuration.py file as given in the
instruction manual to set up the config file.")
    print("Till then, setup.py cannot complete execution. Terminating
early...")
    sys.exit()

```

```

with open(PATH_TO_CONFIG, "r") as f:
    CONFIG = json.load(f)

```

```

if not CONFIG.get("setup_completed"):
    print("Set-up has not been completed before.")
    print("If that is not the case, please run set_configuration and
follow the instructions carefully.")
    sys.exit()

```

```

cbse_logo_png          = Image.open(fp = ASSETS[0]).resize((100, 100))
kvs_logo_jpeg          = Image.open(fp = ASSETS[1]).resize((100, 100))

```

```

HOST = CONFIG.get("host")
USER = CONFIG.get("user")
PASSWD = CONFIG.get("passwd")
DATABASE = CONFIG.get("database")

```

```

CONN_OB = ConnectToMySQL(host=HOST, user=USER, passwd=PASSWD)
CONN_OB.connect_to_database(database_name=DATABASE)

```

```

@atexit.register
def close_connection():
    CONN_OB.close_connection()

```

```

def get_affiliated_school(affiliation_number: int):
    query_to_retrieve_affiliated_school = """
        SELECT
            affiliation_no AS 'Affiliation Number',
            name_of_institution AS 'Name of Institution',
            (CONCAT(state, ', ', district)) AS 'State, District',
            (CONCAT(postal_address, ', ', pin_code)) AS 'Postal
Address',
            website AS 'Website',
            head_of_institution AS 'Head of Institution',
            status_of_the_school AS 'Status of the School',
            school_type AS 'School Type',
            (CONCAT('FROM: ', affiliation_period_start, ' TO: ',
affiliation_period_end)) AS 'Affiliation Period',

```

```

        remarks AS 'Remarks'
    FROM AFFILIATED_SCHOOLS
    WHERE affiliation_no = %s;
    """
    CONN_OB.execute_sql_query(query_to_retrieve_affiliated_school %
affiliation_number)

    fetched_school_records = CONN_OB.fetch_data()

    return fetched_school_records

def get_admit_card_info(exam_roll_number: int):
    query_to_retrieve_admit_card = """
    SELECT
        r.exam_roll_no AS 'Roll No.',
        CONCAT(
            s.affiliation_no, ' - ',
            s.name_of_institution, ', ',
            s.postal_address
        ) AS 'School',
        CONCAT(
            c.affiliation_no, ' - ',
            c.name_of_institution, ', ',
            c.postal_address
        ) AS 'Exam Centre',
        r.grade AS 'CLASS',
        r.candidate_name AS 'Candidate Name',
        r.mother_name AS 'Mother Name',
        r.guardian_name AS 'Guardian''s/Father''s Name',
        r.date_of_birth AS 'Date of Birth',
        r.category_of_pwd AS 'PWD Category',
        r.admit_card_id AS 'Admit Card ID'
    FROM REGISTERED_STUDENTS r
    JOIN AFFILIATED_SCHOOLS s
        ON r.school_affiliation_no = s.affiliation_no
    JOIN AFFILIATED_SCHOOLS c
        ON r.exam_centre_no = c.affiliation_no
    WHERE r.exam_roll_no = %s;
    """
    CONN_OB.execute_sql_query(query_to_retrieve_admit_card %
exam_roll_number)

    fetched_student_records = CONN_OB.fetch_data()

    return fetched_student_records

def get_student_record(exam_roll_number: int):
    query_to_retrieve_student_record = """
    SELECT
        r.candidate_name AS 'Candidate Name',
        r.exam_roll_no AS 'Roll No.',
        r.mother_name AS 'Mother Name',
        r.guardian_name AS 'Guardian''s/Father''s Name',
        r.date_of_birth AS 'Date of Birth',

```

```

        CONCAT(
            s.affiliation_no, ' - ',
            s.name_of_institution, ', ',
            s.postal_address
        ) AS 'School'
    FROM REGISTERED_STUDENTS r
    JOIN AFFILIATED_SCHOOLS s
        ON r.school_affiliation_no = s.affiliation_no
    WHERE r.exam_roll_no = %s;
"""

query_to_retrieve_marksheet = """
SELECT
    c.course_code AS 'Course Code',
    c.course_name AS 'Subject',
    e.marks_theory AS 'Theory',
    (e.marks_practical + e.marks_internal) AS 'IA/PR',
    (e.marks_theory + e.marks_practical + e.marks_internal) AS
'Total'
FROM EXAM_RESULTS e
JOIN COURSES c
    ON e.course_code = c.course_code
JOIN REGISTERED_STUDENTS r
    ON e.exam_roll_no = r.exam_roll_no
WHERE e.exam_roll_no = %s
AND c.grade = r.grade
ORDER BY c.course_code;
"""

CONN_OB.execute_sql_query(query_to_retrieve_student_record %
exam_roll_number)
    fetched_student_record = CONN_OB.fetch_data()

    CONN_OB.execute_sql_query(query_to_retrieve_marksheet %
exam_roll_number)
    fetched_marks_record = CONN_OB.fetch_data()

    return fetched_student_record, fetched_marks_record

class MainWindow:
    def __init__(self, root: tk.Tk):
        self._root = root

        self._root.title("CBSE Database Manager")
        self._root.geometry("800x150")

        self._root.columnconfigure(1, weight=1)

        tk_cbse_logo = ImageTk.PhotoImage(cbse_logo_png)
        tk_kvs_logo = ImageTk.PhotoImage(kvs_logo_jpeg)

        self._photo_1 = tk.Label(
            self._root,
            image=tk_cbse_logo
        )

```

```

self._photo_1.grid(row=0, column=0, padx=10, pady=20)

self.center_text = tk.Label(
    self._root,
    text="Central Board of Secondary Education",
    font=("Jetbrains Mono", 16),
    padx=20, pady=10,
    anchor="center",
    justify="center",
    wraplength=250
)
self.center_text.grid(row=0, column=1, padx=10, pady=20,
sticky="nsew")

self._photo_2 = tk.Label(
    self._root,
    image=tk_kvs_logo
)
self._photo_2.grid(row=0, column=2, padx=10, pady=20)

self._button_frame = tk.Frame(self._root)

self._button_frame.grid(row=0, column=3, padx=10)

self.btn_affl_schl = tk.Button(
    self._button_frame,
    text="Affiliated Schools",
    font=("Jetbrains Mono", 10),
    padx=5, pady=5,
    anchor="center",
    justify="center"
)
self.btn_affl_schl.bind("<Button>", lambda e:
AffiliatedSchools(self._root))

self.btn_admit_card = tk.Button(
    self._button_frame,
    text="Admit Card",
    font=("Jetbrains Mono", 10),
    padx=5, pady=5,
    anchor="center",
    justify="center"
)
self.btn_admit_card.bind("<Button>", lambda e:
AdmitCard(self._root))

self.btn_exam_result = tk.Button(
    self._button_frame,
    text="Exam Result",
    font=("Jetbrains Mono", 10),
    padx=5, pady=5,
    anchor="center",
    justify="center"
)
self.btn_exam_result.bind("<Button>", lambda e:
ExamResult(self._root))

```

```

        self.btn_affl_schl.pack(fill="x", pady=5)
        self.btn_admit_card.pack(fill="x", pady=5)
        self.btn_exam_result.pack(fill="x", pady=5)

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        ...

class AffiliatedSchools(tk.Toplevel):
    def __init__(self, master_window=None):
        super().__init__(master_window)
        self.title("Affiliated Schools")
        self.geometry("500x450")

        self.heading = tk.Label(self, text="Affiliated Schools")
        self.heading.config(font=('Jetbrains Mono', 30))
        self.heading.grid(column=0, row=0, columnspan=2)

        self.entry = tk.Entry(self)
        self.entry.config(font=('Consolas', 30))
        self.entry.grid(column=0, row=1, columnspan=2)

        self.output = tk.Label(self, font=('Jetbrains Mono', 10))
        self.output.grid(column=0, row=5)

        self.button = tk.Button(self, text="SUBMIT", command =
self.func_1)
        self.button.grid(column=2, row=1)

    def func_1(self):
        input_value = self.entry.get()

        if not input_value.isdigit():
            self.output.config(text="Please enter a integer value.")
            return

        affiliation_number = int(input_value)

        data = get_affiliated_school(affiliation_number)

        if data[-1] == 0:
            self.output.config(text="No record found for the given
affiliation number.")
            return

        columns = [desc[0] for desc in data[0]]

        transposed = list(zip(columns, *data[1]))

        display_string = tabulate.tabulate(transposed,
tablefmt="grid")

```

```

        self.output.config(text=display_string)

        return

class AdmitCard(tk.Toplevel):
    def __init__(self, master_window=None):
        super().__init__(master_window)
        self.title("Admit Card")
        self.geometry("400x700")

        self.heading = tk.Label(self, text="Admit Card")
        self.heading.config(font=("Jetbrains Mono", 30))
        self.heading.grid(column=0, row=0, columnspan=2)

        self.entry = tk.Entry(self)
        self.entry.config(font=("Consolas", 30))
        self.entry.grid(column=0, row=1, columnspan=2)

        self.output = tk.Label(self, font=("Jetbrains Mono", 10))
        self.output.grid(column=0, row=5)

        self.button = tk.Button(self, text="SUBMIT",
command=self.func1)
        self.button.grid(column=2, row=1)

    def func1(self):
        input_value = self.entry.get()

        if not input_value.isdigit():
            self.output.config(text="Please enter a integer value")
            return

        exam_roll_no = int(input_value)

        data = get_admit_card_info(exam_roll_no)

        if data[-1] == 0:
            self.output.config(text="No record found.")
            return

        columns = [desc[0] for desc in data[0]]
        transposed = list(zip(columns, *data[1]))

        display_string = tabulate.tabulate(transposed,
tablefmt="grid")

        self.output.config(text=display_string)

        return

class ExamResult(tk.Toplevel):
    def __init__(self, master_window=None):

```

```

super().__init__(master_window)
self.title("Report Maker")
self.geometry("700x300")

self.heading = tk.Label(self, text="Exam Result")
self.heading.config(font=("Jetbrains Mono", 30))
self.heading.grid(column=0, row=0, columnspan=2)

self.entry = tk.Entry(self)
self.entry.config(font=("Consolas", 30))
self.entry.grid(column=0, row=1, columnspan=2)

self.output_frame = tk.Frame(self)
self.output_frame.grid(column=0, row=5, padx=10)

self.output_heading = tk.Label(
    self.output_frame,
    font=("Jetbrains Mono", 10),
    padx=5, pady=5,
    anchor="center",
    justify="center"
)

self.output_student = tk.Label(
    self.output_frame,
    font=("Jetbrains Mono", 10),
    padx=5, pady=5,
    anchor="center",
    justify="center"
)

self.output_marks = tk.Label(
    self.output_frame,
    font=("Jetbrains Mono", 10),
    padx=5, pady=5,
    anchor="center",
    justify="center"
)

self.output_heading.pack(fill="x", pady=5)
self.output_student.pack(fill="x", pady=5)
self.output_marks.pack(fill="x", pady=5)

self.button = tk.Button(self, text="SUBMIT",
command=self.func1)
self.button.grid(column=2, row=1)

def func1(self):
    input_value = self.entry.get()

    if not input_value.isdigit():
        self.output_heading.config(text="Please enter a integer
value.")

```

```

        return

        exam_roll_no = int(input_value)

        student_record, marks_record =
get_student_record(exam_roll_no)

        if student_record[-1] == 0:
            self.output_heading.config(text="No record found.")
            return

        student_record_columns = [desc[0] for desc in
student_record[0]]
        marks_record_columns = [desc[0] for desc in marks_record[0]]

        student_record_transposed = list(zip(student_record_columns,
*student_record[1]))

        display_string_1 =
tabulate.tabulate(student_record_transposed, tablefmt="grid")
        display_string_2 = tabulate.tabulate(marks_record[1],
headers=marks_record_columns, tablefmt="grid")

        self.output_student.config(text=display_string_1)
        self.output_marks.config(text=display_string_2)

        return

def main() -> None:
    root = tk.Tk()
    MainWindow(root)
    root.mainloop()

```

```

if __name__ == "__main__":
    main()

```

src/set_configuration.py

```

import datetime
import json
from pathlib import Path

PATH_TO_CONFIG = Path() / 'config.json'

CURRENT_YEAR: int = datetime.date.today().year

CONFIG = {
    "host": "",
    "user": "",
    "passwd": "",
    "database": "",
    "setup_completed": False
}

```

```

def main():
    if not PATH_TO_CONFIG.exists():
        print("config.json doesn't exists, creating config.json file
first.")
        PATH_TO_CONFIG.touch()
        print(
            f"config.json has been created at the following path:
{PATH_TO_CONFIG.absolute()},\n"
            f"continuing with setting-up your configs."
        )

        host = input("Enter host(press Enter to set host as
'localhost'): ") or "localhost"
        user = input("Enter username(press Enter to set user as
'root'): ") or "root"
        passwd = input("Enter password: ")
        database = "CBSE_EXAM_RESULT_YXXXXX".replace("XXXXX",
str(CURRENT_YEAR))
        setup_completed = bool(input("Press Enter if set up has not been
completed before, else enter 1: "))

        CONFIG["host"] = host
        CONFIG["user"] = user
        CONFIG["passwd"] = passwd
        CONFIG["database"] = database
        CONFIG["setup_completed"] = setup_completed

        with open(PATH_TO_CONFIG, "w") as f:
            json.dump(CONFIG, f, indent=4)

        print("Configuration has been set.")

if __name__ == '__main__':
    main()

```

src/setup.py

```

import datetime
import json
import pickle
import sys
import string
import random
from pathlib import Path
from time import perf_counter

from modules import ConnectToMySQL

from faker import Faker

start_time = perf_counter()

```

```

PATH_TO_AFFILIATED_SCHOOLS_DATA      = Path() / 'data'           /
'affiliated_schools.dat'
PATH_TO_COURSES_DATA                  = Path() / 'data'           /
'courses.dat'
PATH_TO_CONFIG                        = Path() / 'config.json'
PATH_TO_DB_QUERY                      = Path() / 'schemas'        /
'database_schema.sql'

fake = Faker("en_IN")
STUDENT_COUNT: int = 500_000
GRADE_10_STUDENT_COUNT: int = int(STUDENT_COUNT * 0.4)
GRADE_12_STUDENT_COUNT: int = STUDENT_COUNT - GRADE_10_STUDENT_COUNT

def random_dates_for_year(year: int, count: int) ->
list[datetime.date]:
    start = datetime.date(year, 1, 1)
    end = datetime.date(year, 12, 31)
    days = (end - start).days + 1

    result: list[datetime.date] = [
        start + datetime.timedelta(days=random.randrange(days))
        for _ in range(count)
    ]
    result.sort()

    return result

def generate_unique_numbers(digits: int, count: int):
    start = 0
    end = 10**digits - 1
    return random.sample(range(start, end + 1), count)

def generate_unique_admit_card_ids(count: int):
    letters = string.ascii_uppercase
    prefixes = [a + b for a in letters for b in letters]

    result = set()

    while len(result) < count:
        prefix = random.choice(prefixes)
        number = random.randint(100_000, 999_999)
        result.add(f"{prefix}{number:06d}")

    return list(result)

def filter_course_by_grade(course: list, grade: int) -> list:
    return list(
        filter(
            lambda x: x[-1] == grade,
            course

```

```

    )
)

def pick_courses(grade: int):
    main = random.sample(courses[grade]["main"], 5)
    additional = random.sample(courses[grade]["additional"], 2)

    return main, additional

def generate_marks(course):
    course_number, course_name, th_max, pr_max, ia_max, grade = course

    th = random.randint(0, th_max)
    pr = random.randint(0, pr_max)
    ia = random.randint(0, ia_max)

    return th, pr, ia

if not PATH_TO_AFFILIATED_SCHOOLS_DATA.exists():
    print(f"Affiliated schools data doesn't exists at: {PATH_TO_AFFILIATED_SCHOOLS_DATA.absolute()}")
    print("Either download the affiliated_schools.dat from the GitHub repo at: ...")
    print("or use the affiliated_schools.dat in the backup directory.")
    print("Till then, setup.py cannot complete execution. Terminating early...")
    sys.exit()

if not PATH_TO_COURSES_DATA.exists():
    print(f"Courses data file doesn't exists at: {PATH_TO_COURSES_DATA.absolute()}")
    print("Either download the courses.dat from the GitHub repo at: ...")
    print("or run scripts/parse_courses.py according to the instructions given in the manual to generate the file.")
    print("Till then, setup.py cannot complete execution. Terminating early...")
    sys.exit()

if not PATH_TO_CONFIG.exists():
    print(f"Config file doesn't exists at the following file path: {PATH_TO_CONFIG.absolute()}")
    print("Run the set_configuration.py file as given in the instruction manual to set up the config file.")
    print("Till then, setup.py cannot complete execution. Terminating early...")
    sys.exit()

if not PATH_TO_DB_QUERY.exists():

```

```

        print(f"Database query file doesn't exists at:
{PATH_TO_DB_QUERY.absolute()}")
        print("Download the SQL file from the GitHub repo at: ...")
        print("Till then, setup.py cannot complete execution. Terminating
early...")
        sys.exit()

with (
    open(PATH_TO_AFFILIATED_SCHOOLS_DATA, "rb") as f1,
    open(PATH_TO_COURSES_DATA, "rb") as f2,
    open(PATH_TO_CONFIG, 'r') as f3,
    open(PATH_TO_DB_QUERY, 'r') as f4
):
    AFFILIATED_SCHOOLS    = pickle.load(f1)
    COURSES_DATA          = pickle.load(f2)
    CONFIG                = json.load(f3)
    QUERY                 = f4.read()

if CONFIG.get("setup_completed"):
    print("Set-up has already been completed before.")
    print("If that is not the case, please run set_configuration and
follow the instructions carefully.")
    sys.exit()

HOST    = CONFIG.get("host")
USER    = CONFIG.get("user")
PASSWD  = CONFIG.get("passwd")

CURRENT_YEAR: int    = datetime.date.today().year
QUERY        = QUERY.replace("XXXX", str(CURRENT_YEAR))

MAIN_COURSES          = list(
    map(
        lambda x: (*x[:-2], x[-1]),
        filter(
            lambda x: x[-2]=='MAIN',
            COURSES_DATA
        )
    )
)

ADDITIONAL_COURSES    = list(
    map(
        lambda x: (*x[:-2], x[-1]),
        filter(
            lambda x: x[-2]=='ADDITIONAL',
            COURSES_DATA
        )
    )
)

courses = {
    10: {

```

```

        "main": filter_course_by_grade(MAIN_COURSES, 10),
        "additional": filter_course_by_grade(ADDITIONAL_COURSES, 10)
    },
    12: {
        "main": filter_course_by_grade(MAIN_COURSES, 12),
        "additional": filter_course_by_grade(ADDITIONAL_COURSES, 12)
    }
}

SECONDARY_SCHOOLS_AFFILIATION_NUMBERS = [
    row[0] for row in
    filter(
        lambda x: x[-5].strip() == "Secondary Level",
        AFFILIATED_SCHOOLS
    )
]
SENIOR_SECONDARY_SCHOOLS_AFFILIATION_NUMBERS = [
    row[0] for row in
    filter(
        lambda x: x[-5].strip() == "Senior Secondary Level",
        AFFILIATED_SCHOOLS
    )
]

# Generating required number of affiliation numbers for students
GRADE_10_SCHOOL_AFFILIATION_NOS = random.choices(
    population=SECONDARY_SCHOOLS_AFFILIATION_NUMBERS,
    k=GRADE_10_STUDENT_COUNT
)
GRADE_12_SCHOOL_AFFILIATION_NOS = random.choices(
    population=SENIOR_SECONDARY_SCHOOLS_AFFILIATION_NUMBERS,
    k=GRADE_12_STUDENT_COUNT
)

GRADE_10_DATE_OF_BIRTHS = random_dates_for_year(CURRENT_YEAR - 16,
GRADE_10_STUDENT_COUNT)
GRADE_12_DATE_OF_BIRTHS = random_dates_for_year(CURRENT_YEAR - 18,
GRADE_12_STUDENT_COUNT)

EXAM_ROLL_NUMBERS = random.sample(range(1_000_000, 9_999_999),
STUDENT_COUNT)
EXAM_ROLL_NUMBERS.sort()
AFFILIATION_NUMBERS = GRADE_10_SCHOOL_AFFILIATION_NOS +
GRADE_12_SCHOOL_AFFILIATION_NOS
EXAM_CENTRE_NOS = random.choices(
    population=AFFILIATION_NUMBERS,
    k=STUDENT_COUNT
)
CANDIDATE_NAMES = [fake.name() for _ in range(STUDENT_COUNT)]
DATE_OF_BIRTHS = GRADE_10_DATE_OF_BIRTHS + GRADE_12_DATE_OF_BIRTHS
GENDERS = random.choices(
    population=['M', 'F', 'O'],

```

```

        weights=[0.52, 0.47, 0.01],
        k=STUDENT_COUNT
    )
    GRADES = [10] * GRADE_10_STUDENT_COUNT + [12] * GRADE_12_STUDENT_COUNT
    MOTHER_NAMES = [fake.name_female() for _ in range(STUDENT_COUNT)]
    GUARDIAN_NAMES = [fake.name_male() for _ in range(STUDENT_COUNT)]
    AADHAR_NOS = generate_unique_numbers(12, STUDENT_COUNT)
    APAAR_ID = generate_unique_numbers(12, STUDENT_COUNT)
    CATEGORY_OF_PWD = random.choices(
        population=['Y', 'N'],
        weights=[0.01, 0.99],
        k=STUDENT_COUNT
    )
    ADMIT_CARD_IDS = generate_unique_admit_card_ids(STUDENT_COUNT)

REGISTERED_STUDENTS = list(
    zip(
        EXAM_ROLL_NUMBERS,
        AFFILIATION_NUMBERS,
        EXAM_CENTRE_NOS,
        CANDIDATE_NAMES,
        DATE_OF_BIRTHS,
        GENDERS,
        GRADES,
        MOTHER_NAMES,
        GUARDIAN_NAMES,
        AADHAR_NOS,
        APAAR_ID,
        CATEGORY_OF_PWD,
        ADMIT_CARD_IDS
    )
)

STUDENTS = list(
    # [(roll number, grade), ...]
    map(
        lambda x: (x[0], x[6]),
        REGISTERED_STUDENTS
    )
)

EXAM_RESULTS = []

for roll_no, grade in STUDENTS:
    main_courses, add_courses = pick_courses(grade)

    for course in main_courses:
        th, pr, ia = generate_marks(course)
        EXAM_RESULTS.append(
            (roll_no, course[0], th, pr, ia)
        )

    for course in add_courses:

```

```

        th, pr, ia = generate_marks(course)
        EXAM_RESULTS.append(
            (roll_no, course[0], th, pr, ia)
        )

STUDENT_BATCH = 5000
RESULT_BATCH = 2000

with ConnectToMySQL(
    host=HOST, user=USER, passwd =PASSWD
) as conn_ob:
    for line in QUERY.split(";"):
        stmt = line.strip()
        if stmt:
            conn_ob.execute_sql_query(stmt)
    print("\nDatabase has been created.")

    conn_ob.insert_data("INSERT INTO COURSES VALUES (%s, %s, %s, %s,
%s, %s);", (MAIN_COURSES + ADDITIONAL_COURSES))
    conn_ob.commit_to_database()
    print("\nCourses has been added.")

    conn_ob.insert_data("INSERT INTO AFFILIATED_SCHOOLS VALUES (%s,%s,
%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s);", AFFILIATED_SCHOOLS)
    conn_ob.commit_to_database()
    print("\nAffiliated schools has been added.")

    for i in range(0, STUDENT_COUNT, STUDENT_BATCH):
        chunk = REGISTERED_STUDENTS[i:i+STUDENT_BATCH]
        conn_ob.insert_data("INSERT INTO REGISTERED_STUDENTS VALUES
(%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)", chunk)
        conn_ob.commit_to_database(silent=True)
        print("\nRegistered students added.")

    for i in range(0, len(EXAM_RESULTS), RESULT_BATCH):
        chunk = EXAM_RESULTS[i:i+RESULT_BATCH]
        conn_ob.insert_data("INSERT INTO EXAM_RESULTS VALUES (%s,%s,
%s,%s,%s)", chunk)
        conn_ob.commit_to_database(silent=True)
        print("\nExam results added.")

CONFIG["setup_completed"] = True
with open(PATH_TO_CONFIG, "w") as f:
    json.dump(CONFIG, f, indent=4)

print("Set up has been completed. Now you can run main.py to start the
program.")

end_time = perf_counter()
print(f"Total time taken: {end_time - start_time:.3f}s")

```

schemas/database_schema.sql

```
CREATE DATABASE CBSE_EXAM_RESULT_YXXXX;  
USE CBSE_EXAM_RESULT_YXXXX;
```

```
CREATE TABLE COURSES(  
    course_code SMALLINT UNSIGNED,  
    course_name VARCHAR(50) NOT NULL,  
  
    marks_theory TINYINT UNSIGNED NOT NULL,  
    marks_practical_assessment TINYINT UNSIGNED DEFAULT 0 NOT NULL,  
    marks_internal_assessment TINYINT UNSIGNED DEFAULT 0 NOT NULL,  
  
    grade TINYINT UNSIGNED NOT NULL  
        CHECK (grade IN (10, 12)),  
  
    PRIMARY KEY (course_code, grade),  
  
    CONSTRAINT chk_course_valid_marks CHECK(  
        (  
            marks_theory BETWEEN 0 AND 100  
            AND marks_practical_assessment BETWEEN 0 AND 100  
            AND marks_internal_assessment BETWEEN 0 AND 100  
        )  
        AND -- sum is always 100  
        (  
            marks_theory + marks_practical_assessment +  
marks_internal_assessment = 100  
        )  
    );
```

```
CREATE TABLE AFFILIATED_SCHOOLS(  
    affiliation_no INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
    name_of_institution VARCHAR(80) NOT NULL,  
  
    state VARCHAR(40) NOT NULL,  
    district VARCHAR(40) NOT NULL,  
    postal_address VARCHAR(255) NOT NULL,  
    pin_code MEDIUMINT UNSIGNED NOT NULL,  
  
    website VARCHAR(100) DEFAULT '' NOT NULL,  
  
    year_of_foundation YEAR NOT NULL,  
    date_of_first_opening_of_school DATE NOT NULL,  
  
    head_of_institution VARCHAR(50) NOT NULL,  
  
    status_of_the_school ENUM('Middle Class', 'Secondary Level',  
'Senior Secondary Level')  
        DEFAULT 'Senior Secondary Level' NOT NULL,
```

```

        school_type ENUM('KV', 'GOVT', 'JNV', 'STSS', 'INDEPENDENT')
DEFAULT 'INDEPENDENT' NOT NULL,
        affiliation_period_start DATE NOT NULL,
        affiliation_period_end DATE NOT NULL,

        remarks VARCHAR(255) DEFAULT '' NOT NULL,

        CONSTRAINT chk_pin_code CHECK (pin_code BETWEEN 100000 AND 999999)
);

```

```

CREATE TABLE REGISTERED_STUDENTS(

        exam_roll_no INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,

        school_affiliation_no INT UNSIGNED NOT NULL,
        exam_centre_no INT UNSIGNED NOT NULL,

        candidate_name VARCHAR(60) NOT NULL,
        date_of_birth DATE NOT NULL,
        gender ENUM('M', 'F', 'O') NOT NULL,
        grade TINYINT UNSIGNED CHECK (grade IN (10,12)),

        mother_name VARCHAR(60) NOT NULL,
        guardian_name VARCHAR(60) NOT NULL,

        aadhar_no BIGINT UNSIGNED UNIQUE NOT NULL,
        apaar_id BIGINT UNSIGNED UNIQUE NOT NULL,

        category_of_pwd ENUM('Y', 'N') NOT NULL,
        -- admit card starts with 2 english letters and rest filled with
integers from 0 to 9
        admit_card_id CHAR(8) UNIQUE NOT NULL,

        FOREIGN KEY (school_affiliation_no)
            REFERENCES AFFILIATED_SCHOOLS(affiliation_no),
        FOREIGN KEY (exam_centre_no)
            REFERENCES AFFILIATED_SCHOOLS(affiliation_no),

        CONSTRAINT chk_aadhar_no
            CHECK (aadhar_no BETWEEN 0 AND 999999999999),
        CONSTRAINT chk_apaar_id
            CHECK (apaar_id BETWEEN 0 AND 999999999999),
        CONSTRAINT chk_admit_card_id
            CHECK (admit_card_id REGEXP '[A-Z]{2}[0-9]{6}')
);

```

```

CREATE TABLE EXAM_RESULTS (
        exam_roll_no INT UNSIGNED NOT NULL,
        course_code SMALLINT UNSIGNED NOT NULL,

        marks_theory TINYINT UNSIGNED NOT NULL,
        marks_practical TINYINT UNSIGNED DEFAULT 0 NOT NULL,
        marks_internal TINYINT UNSIGNED DEFAULT 0 NOT NULL,

```

```
PRIMARY KEY (exam_roll_no, course_code),  
  
FOREIGN KEY (exam_roll_no)  
    REFERENCES REGISTERED_STUDENTS(exam_roll_no)  
    ON DELETE CASCADE,  
  
FOREIGN KEY (course_code)  
    REFERENCES COURSES(course_code),  
  
CONSTRAINT chk_result_valid_marks CHECK(  
    marks_theory BETWEEN 0 AND 100  
    AND marks_practical BETWEEN 0 AND 100  
    AND marks_internal BETWEEN 0 AND 100  
    )  
);
```

Snippets of Test Run

CBSE Database Manager

Central Board of
Secondary Education

Affiliated Schools

Admit Card

Exam Result

Affiliated Schools

Affiliated Schools

3430036

SUBMIT

Affiliation Number	3430036
Name of Institution	CENTRAL ACADEMY
State, District	JHARKHAND, RANCHI
Postal Address	BARIATU ROAD RANCHI JHARKHAND, 834009
Website	www.mgmschoolitarsi.com
Head of Institution	SUTAPA BHATTACHARIYA
Status of the School	Senior Secondary Level
School Type	INDEPENDENT
Affiliation Period	FROM: 2024-04-01 TO: 2029-03-31
Remarks	

Admit Card

Admit Card

9997599

SUBMIT

Roll No.	9997599
School	3430039 - G & H HIGH SCHOOL, JAIL ROAD EAST DISTT RANCHI, JHARKHAND
Exam Centre	2430465 - CHURCH BELL SCHOOL, T.N MUKHERJEE ROAD, RAGHUNATHPUR - III, P. S. DANKUNI
CLASS	12
Candidate Name	Onkar Mitra
Mother Name	Jasmit Kant
Guardian's/Father's Name	Utkarsh Parekh
Date of Birth	2007-12-31
PWD Category	N
Admit Card ID	ZY964357

Report Maker

Exam Result

9997599

SUBMIT

No record found.

Candidate Name	Onkar Mitra
Roll No.	9997599
Mother Name	Jasmit Kant
Guardian's/Father's Name	Utkarsh Parekh
Date of Birth	2007-12-31
School	3430039 - G & H HIGH SCHOOL, JAIL ROAD EAST DISTT RANCHI, JHARKHAND

Course Code	Subject	Theory	IA/PR	Total
2	Hindi Elective	26	1	27
3	Urdu Elective	69	16	85
28	Political Science	64	13	77
43	Chemistry	12	27	39
105	Bengali	40	3	43
830	Design	5	13	18
845	Physical Activity Trainer	35	44	79

****Please note that the font and text size were changed for presentation purposes only, actual program may vary****

User Manual

To run the project, the user must first ensure that Python and the UV package manager are installed on their system(**Appendix A.3** and **Appendix A.4**, respectively).

All Python programs in this project must be executed from the root directory using module execution syntax, and in the terminal(**Appendix A.2**).

All outputs, including affiliated school searches, admit cards, and examination results, are displayed directly within the graphical user interface in tabular form.

Initial Setup

1. Clone or extract the project repository to a local directory, and open the project folder in the code editor(**Appendix A.2**).
2. Install the UV package manager and initialize the virtual environment(**Appendix A.4** and **Appendix A.5**, respectively).
3. **(Important)** Activate the virtual environment(**Appendix B.5**).
4. Run the configuration script to create and manage the config.json file(**Appendix B.1**):
 - `~$ python -m src.set_configuration`
5. Run the setup script to initialize the database and generate data(**Appendix B.2**):
 - `~$ python -m src.setup`

Running the Application

After setup is complete, the main application can be launched using(**Appendix B.3**):

```
~$ python -m main
```

Optional Data Updates

Refer to **Appendix B.4** if the user wishes to add or update affiliated school data or course data.

Brief Background of Organization

...

Reference

1. CBSE main website: <https://www.cbse.gov.in>
2. List of affiliated schools: <https://saras.cbse.gov.in/SARAS/AffiliatedList/ListOfSchdirReport>
3. MySQL integer(INT) types for database schema: <https://dev.mysql.com/doc/refman/8.4/en/integer-types.html>
4. CBSE Result page: <https://dev.mysql.com/doc/refman/8.4/en/integer-types.html>
5. Year-wise summary of number of students who appeared for the Class 12 exams: <https://www.timesnownews.com/education/cbse-board-results-2025-number-of-students-who-appeared-for-cbse-class-12-exam-since-2017-article-151431303>
6. Year-wise summary of number of students who appeared for the Class 10 exams: <https://www.timesnownews.com/education/cbse-class-10th-result-2025-pdf-download-year-wise-pass-percentage-trends-2017-to-2024-latest-announcement-article-151405464>
7. VS Code download page: <https://code.visualstudio.com/download>
8. VS Code setup page for Windows: <https://code.visualstudio.com/docs/setup/windows>
9. VS Code setup page for Linux: <https://code.visualstudio.com/docs/setup/linux>
10. UV Python package manager: <https://docs.astral.sh/uv/>

Appendix A: Environment and Dependency Setup

This appendix describes the one-time environment setup required to run the project. These steps are not part of normal application usage and are intended to be performed during initial installation.

All commands must be executed in the terminal from the root directory of the project(Appendix A.2).

The text following the `~$` symbol indicates the command prompt to be written in the terminal, and the symbol itself should not be typed.

A.1 Supported Platforms

The project is designed to be cross-platform and can be executed on:

- Windows 10 and Windows 11
- Any Linux distribution with Python support

The project has not been formally tested on macOS, but no platform-specific dependencies are used.

A.2 Code Editor and the Terminal

Any code editor can be used. However, Visual Studio Code is strongly recommended.

On Windows:

Download the Visual Studio Code installer for Windows from the following link:
<https://code.visualstudio.com/download>

Once the download finishes, run the installer to install Visual Studio Code on your system.

On Linux and macOS:

Refer to <https://code.visualstudio.com/docs/> for your OS-specific installation instructions.

Opening the Terminal in the root directory of the project

Navigate to the following GitHub repository:
<https://github.com/BulkTornado/CBSE-RDMS-Project>, and download the zip archive as instructed there. Extract the

project folder to a local directory. Open VS Code, and from the menu, use the **File > Open Folder** menu command or press **Ctrl + O**, then open the project folder by navigating to it on your PC.

Next, open the terminal in any of the following ways:

- From the menu, use the **Terminal > New Terminal** or **View > Terminal** menu commands.
- From the **Command Palette**(**Ctrl + Shift + P**), type **View: Toggle Terminal** command.

A.3 Python Installation

Ensure that Python is installed and accessible via the `python` command in the terminal(**Appendix A.2**).

The recommended method is to install Python from the official Python distribution appropriate for the operating system. After installation, verify by typing the following in the terminal:

```
~$ python --version
```

A.4 Installing the UV Python Package Manager

The project uses UV to manage the virtual environments and dependencies.

On Windows(Powershell):

```
~$ powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

On Linux and macOS:

```
~$ curl -LsSF https://astral.sh/uv/install.sh | sh
```

After installation, verify UV availability using:

```
~$ uv --version
```

A.5 Setting Up the Project Environment

Navigate to the root directory of the project and initialize the virtual environment and dependencies:

```
~$ uv init
```

UV automatically creates a `.venv` directory and installs all required dependencies as specified by the project configuration.

To verify installed dependencies, run:

```
~$ uv tree
```

This should display a dependency tree of all installed package.

Appendix B: Command Reference

This appendix lists all terminal commands used in the project along with their purpose.

All commands must be executed in the terminal from the root directory of the project(Appendix A.2).

The text following the ~\$ symbol indicates the command prompt to be written in the terminal, and the symbol itself should not be typed.

B.1 Configuration

```
~$ python -m src.set_configuration
```

Purpose:

- Creates or updates the config.json file
- Stores environment-specific and database-related configuration values.

B.2 Database Initialization and Data Generation

```
~$ python -m src.setup
```

Purpose:

- Creates database schema
- Initializes tables
- Generates mock data for affiliated schools, courses, students, and examination results.

B.3 Running the Main Application

```
~$ python -m main
```

Purpose:

- Launches the graphical user interface
- Enables interaction with the database through predefined operations

B.4 Optional Data Processing Scripts

```
~$ python -m src.parse_affiliated_school_data
```

Purpose:

- Parses and updates affiliated school data from source datasets

```
~$ python -m src.parse_courses
```

Purpose:

- Parses and updates course-related data

These scripts may be executed independently but require dropping the database from MySQL(**Appendix B.5**) if it has already been generated, re-running the the configuration script(**Appendix B.1**), and then re-running the set-up script(**Appendix B.2**).

B.5 Activating the Virtual Environment

On Windows:

```
~$ source .venv\bin\activate
```

On Linux and macOS:

```
~$ source .venv/bin/activate
```

B.6 Dropping Database from MySQL

```
~$ DROP DATABASE IF EXISTS CBSE_EXAM_RESULTS_YXXXX;
```

Purpose:

- Drop the database from MySQL if it exists

